EESC4550/5550: Practical Atmospheric Modeling

Topic 2: Linux Computing for Beginners

Prof. Shixian ZHAI

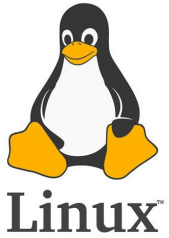
2026/27 Term 1





DEPARTMENT OF
EARTH AND ENVIRONMENTAL SCIENCES
地 球 與 環 境 科 學 系





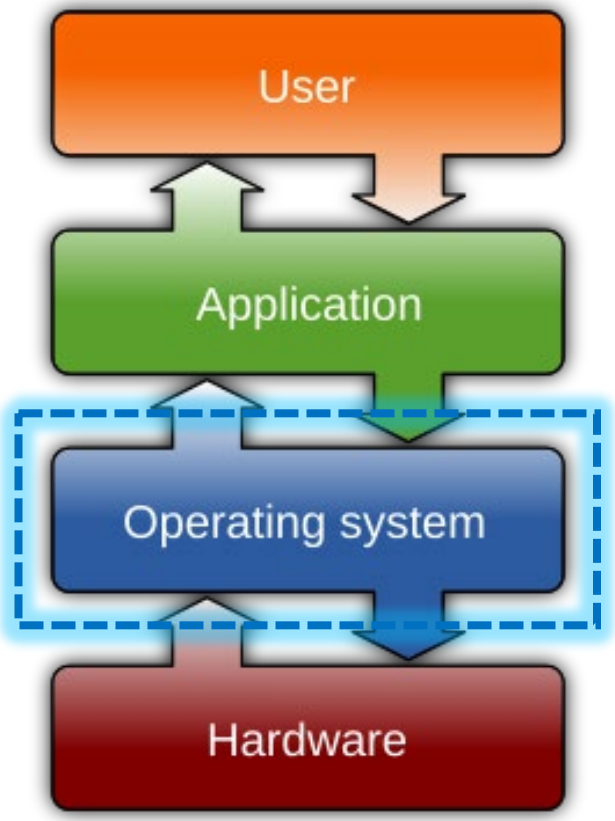
Prepared by Hok Fong WONG

Outline of This Topic

1. Purpose of an Operating System
2. Introducing Linux
3. Establishing a Connection
4. Command Line Interface
5. The File System
6. Manipulating File Contents
7. Users, Groups and Access Control
8. Processes
9. Remote Development on Visual Studio Code (Optional)

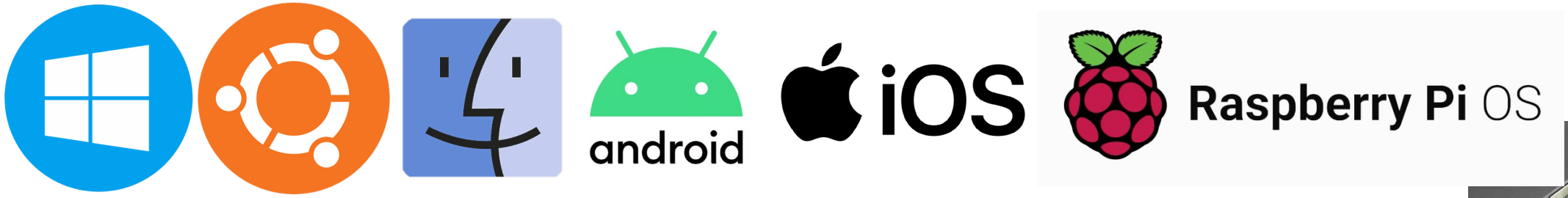
Purpose of an Operating System

- An **operating system (OS)** is a layer of "privileged" software that manages computer hardware and software resources.
- **Core functions of an OS:**
  - **Resource management**: allocates the resources (memory, CPU time, disk space) they request, reclaims the allocated resources or halts their execution.
  - **Device control**: Provides drivers and protocols to handle the communications (input/output) with peripheral devices (screen, keyboard, mouse, network, printer, ...);
  - **File management**: Organizes and tracks how data is stored, retrieved, and secured on devices

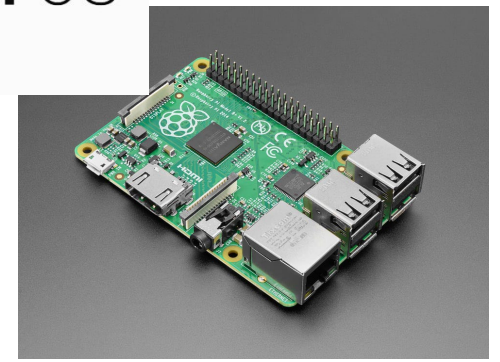


Types and Examples of Operating Systems

- Desktop OS: Windows, Ubuntu, MacOS
- Mobile OS: Android, iOS
- Embedded OS: Raspberry Pi OS

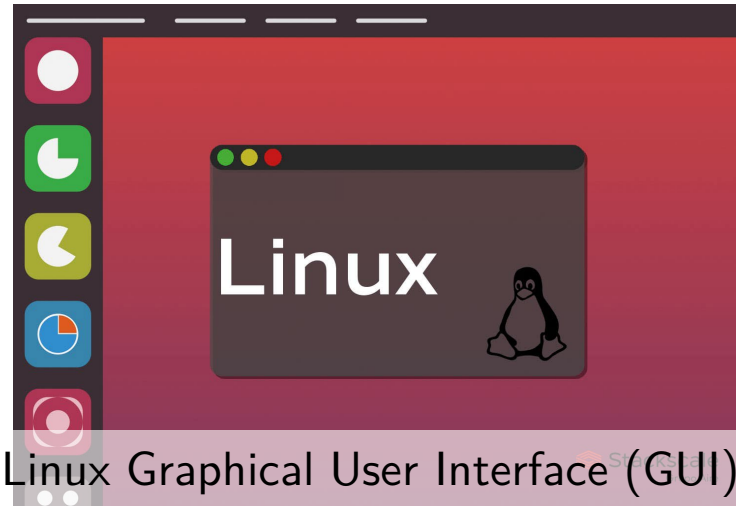


An **embedded system** is a purpose-built computing system. Common applications include automotive control systems (antilock brakes), industrial automation, medical equipment (pacemakers), and consumer electronics (washing machines, microwave ovens).

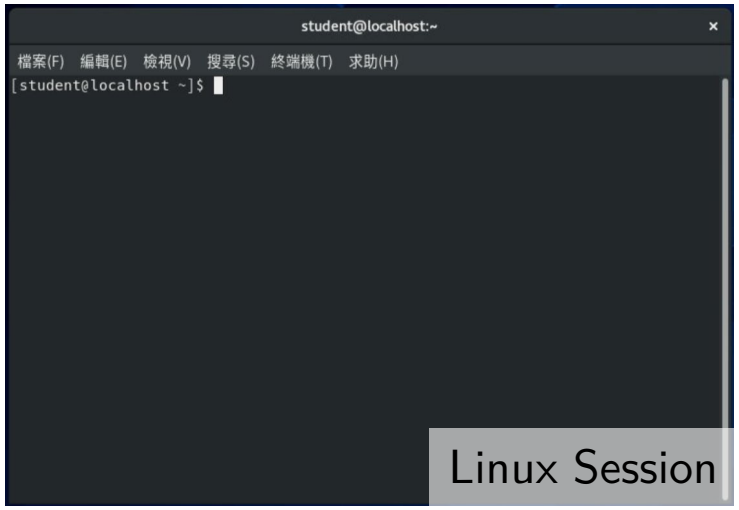


Why Linux?

- Pre-emptive multitasking & multi-user system, ensuring every process gets a fair share of the CPU and resource security among different users
- Open-source, backed by an international community of programmers
- *Free* (freely copied, modified, and redistributed)
- Lightweight, durable, (relatively) secure and stable with track record



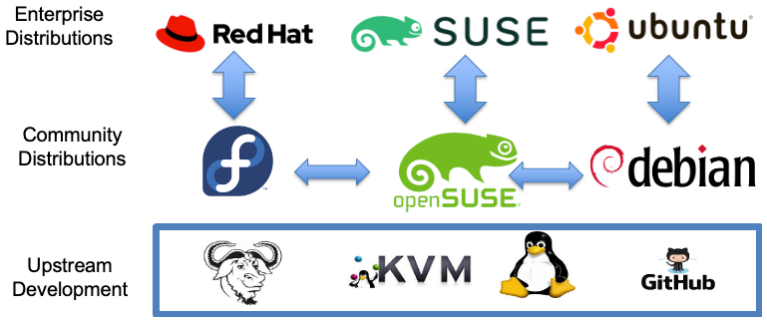
Linux Graphical User Interface (GUI)

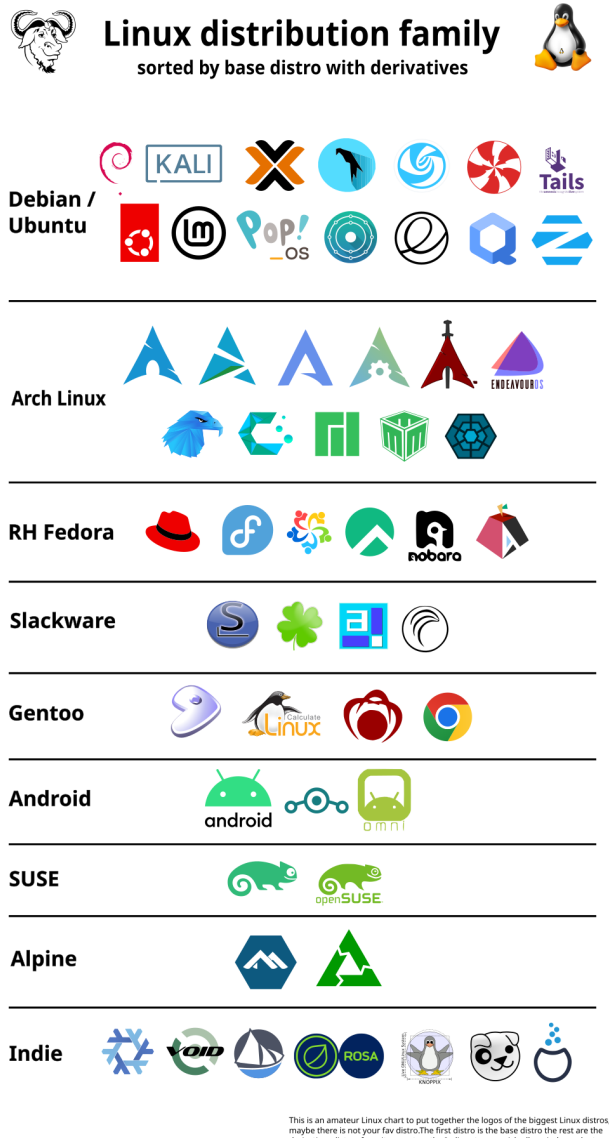


Linux Session

Linux Distributions

- A Linux distribution is a **bundle** that consists of a Linux **kernel** and a selection of **prepackaged software**, **and administration tools** facilitating the installation and update of these packages.
- Varies between *community-based* and *commercial-based support*, and the *licenses* for the software they provide.



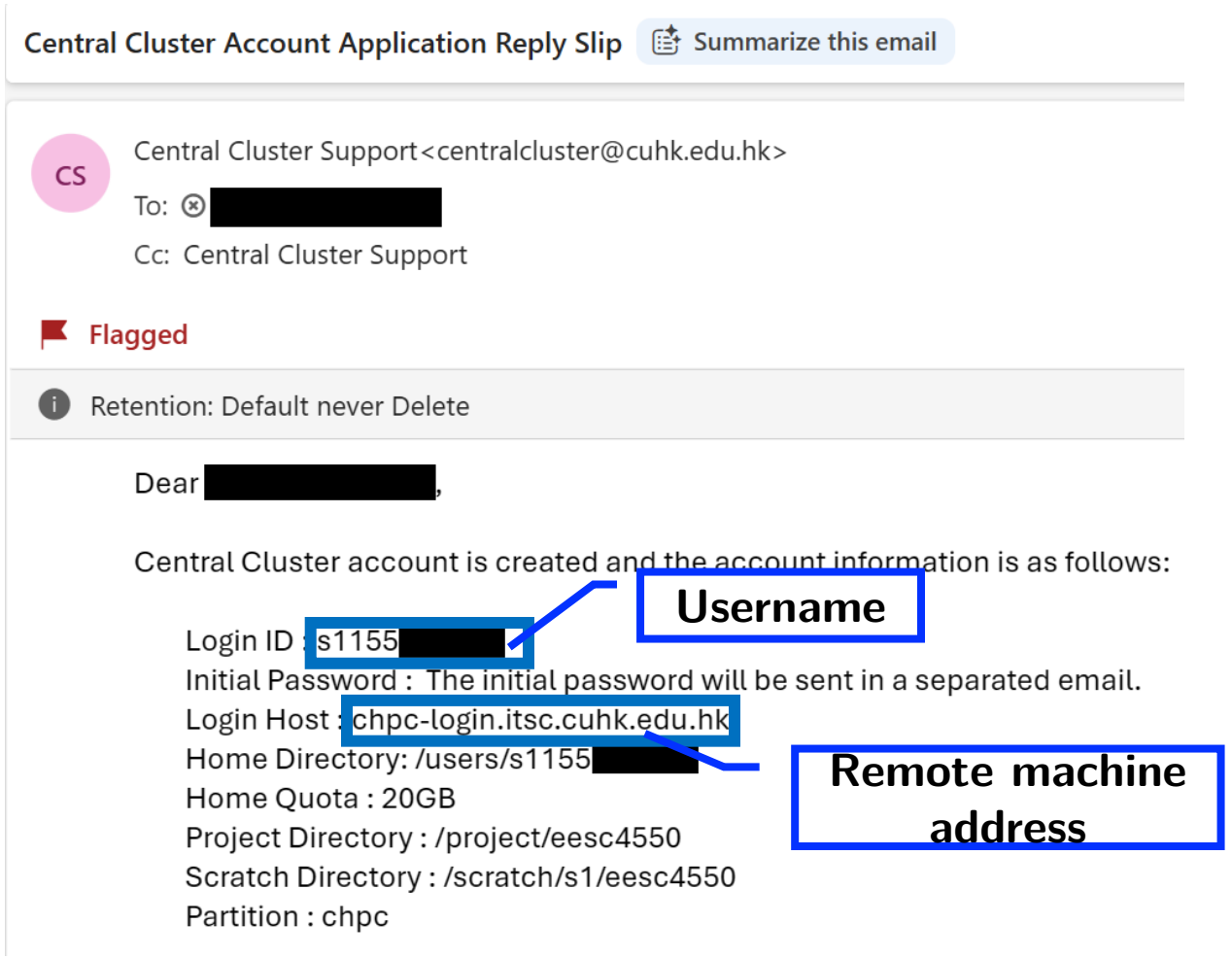


Connecting to the Central Cluster

Step 0. You should have received an email containing the information required to connect to your central cluster account.

To establish a connection, we will use the **Secure Shell (SSH) Protocol**:

- Opens an encrypted channel allowing information exchange between two machines.
- Requires knowledge of the **address** of the remote machine and the **login ID** (username) on the remote machine.

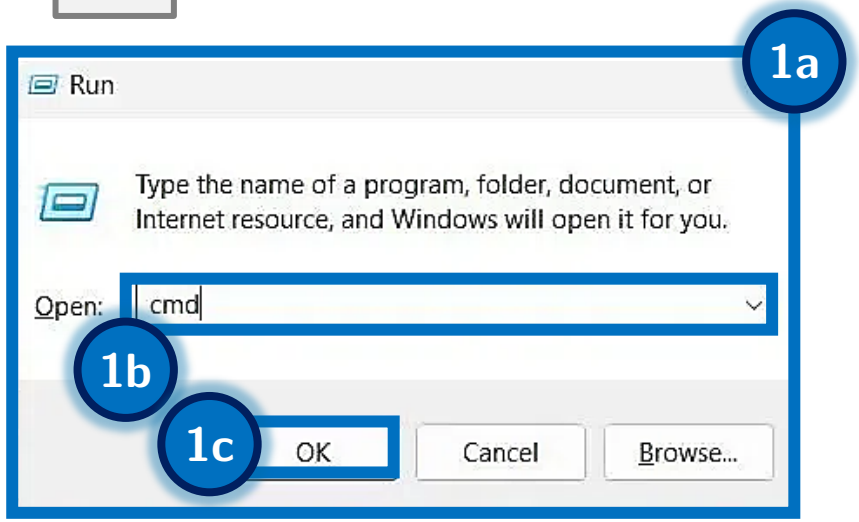


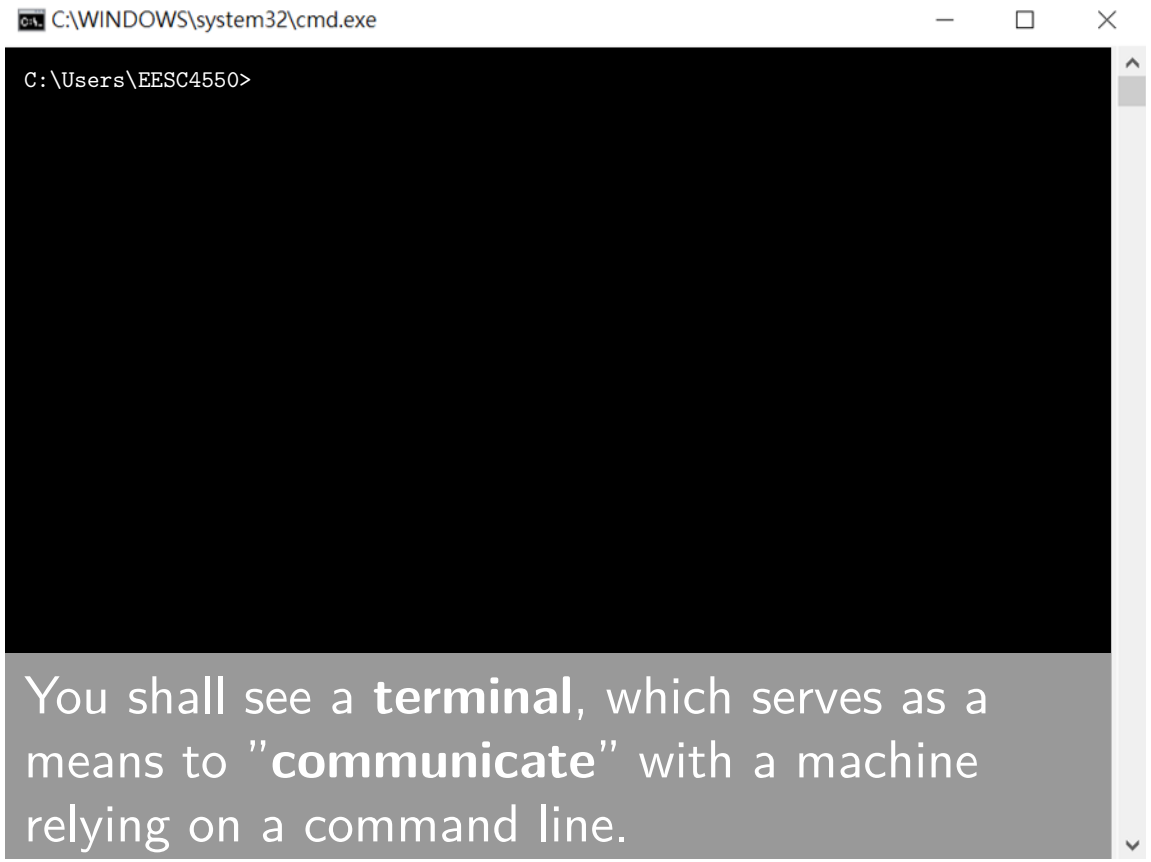
Connecting to the Central Cluster - Starting the Terminal (Windows Users)

Step 1a. Press `Win` + `R` to open "**Run**" Dialog

Step 1b. Type `cmd`

Step 1c. Press `Enter` or the button `OK` .



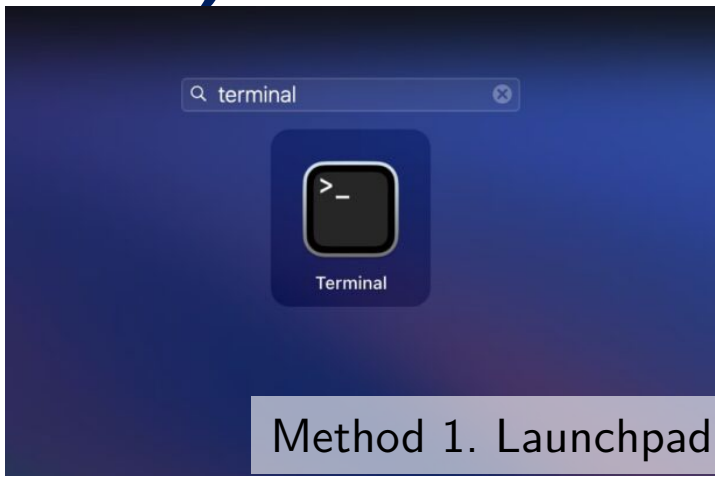


You shall see a **terminal**, which serves as a means to "**communicate**" with a machine relying on a command line.

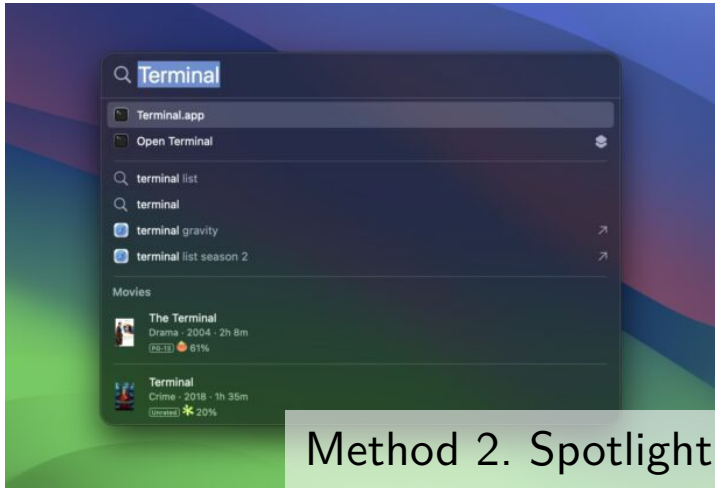
Connecting to the Central Cluster - Starting the Terminal (MacOS Users)

Step 1. There are two methods to start the terminal on MacOS.

- **Launchpad**: Click the **Launchpad** icon in the Dock, type **"Terminal"**, and open it.
- **Spotlight**: Press `Command (⌘)` + `Space` , type **"Terminal"**, and hit `Enter` .



Method 1. Launchpad



Method 2. Spotlight

Connecting to the Central Cluster

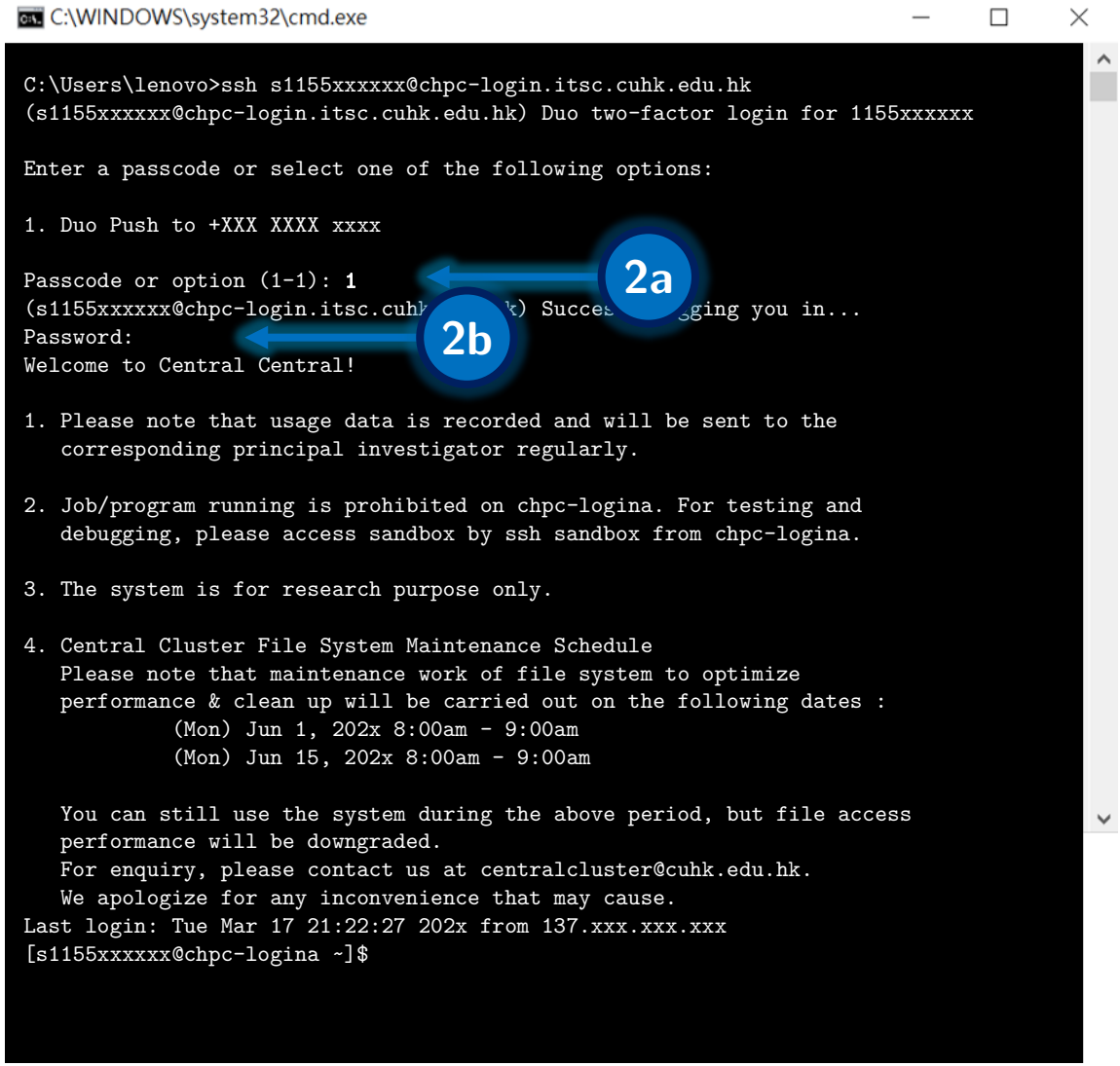
Step 2. Type the following command

`ssh <your-computing-id>@<remote-address>`.

You will then receive a notification on your mobile device for the two-factor authentication. Confirm this login attempt.

Then, you will be prompted to enter the password. Press `Enter` when you finish typing.

Note. When typing your password, you may realize that nothing new is shown on the screen. This is normal, as Linux does so to prevent anyone from seeing the length of your password. If you think you have messed up, press `Ctrl` + `U` to clear the entire line.



Structure of the Command Line

Terminal

```
[s1155xxxxxx@chpc-logina ~]$ head -n 20 eesc4550.log # print the first 20 lines
```

Component	Meaning
[s1155xxxxxx@chpc-logina ~]$	The *prompt*: displays the current user's login (**s1155xxxxxx**), the host (or machine) name (**chpc-logina**), the current directory (working directory) (**~**)
head	The name of the program to run (first word following the prompt)
-n 20	A command option (**-n**) possibly with an associated value (**20**).
eesc4550.log	A command argument (to use the file named **eesc4550.log**)
# print the first 20 lines	Single line comments (starting with a # and will be ignored)

Note. You may wonder why we are not operating on the GUI. It makes sense that you are not enjoying how unintuitive it is, but there are two practical reasons.

1. For experienced users, typing a command is faster than navigating through multiple menus, dialog boxes, and windows. For repetitive tasks, terminal commands can be saved into scripts for automation.
2. Terminals consume far less RAM and CPU than graphical desktop environments with minimal latency.

Some Notes for the Command Line



- The **space character** is used to **separate** the various fields of the command line.
- The **character case (upper/lower) is important** (`head` is not the same as `HEAD`).
- Each command has its own set of arguments and options.
- The **Enter key** (⏎) is used to run the program head.
- ~ means the shell is in the current user's home directory.
- **Trick**: you may press the `Tab` key once to **autocomplete** the file or folder name; and **twice** if you want to list all possible matches starting with the characters you typed.

Understanding a Command with Bulit-in Help

- Each command bundled with the standard system is *documented*.
- To display a short documentation on how to use a command, add the `--help` or `-h` options.

Terminal

```
[s1155xxxxxx@chpc-logina ~]$ ls --help
Usage: ls [OPTION]... [FILE]...
List information about the FILEs (the current directory by default).
Sort entries alphabetically if none of -cftuvSUX nor --sort is specified.

Mandatory arguments to long options are mandatory for short options too.
  -a, --all                  do not ignore entries starting with .
  -A, --almost-all           do not list implied . and ..
      --author               with -l, print the author of each file
  ...
```

Understanding a Command with Bulit-in Help

- For a more detailed documentation, it is possible to use the `man` (i.e. ***man**ual*) command. Giving it as an argument the name of the command for which to display the documentation.

Terminal

```
[s1155xxxxxx@chpc-logina ~]$ man ls
LS(1)                          User Commands                          LS(1)

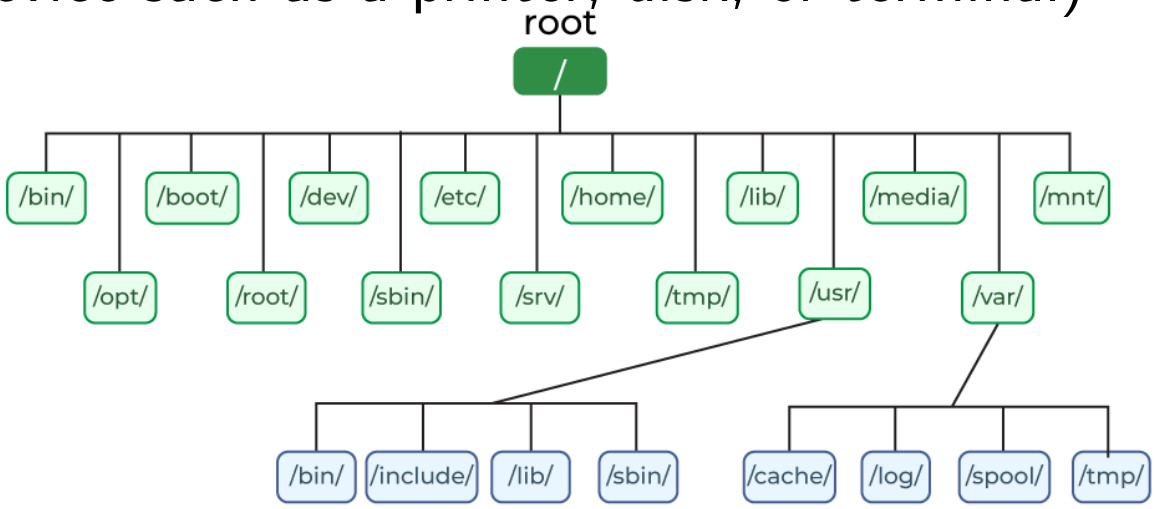
NAME
       ls - list directory contents

SYNOPSIS
       ls [OPTION]... [FILE]...

DESCRIPTION
       List information about the FILEs (the current directory by default).  Sort entries
...
```

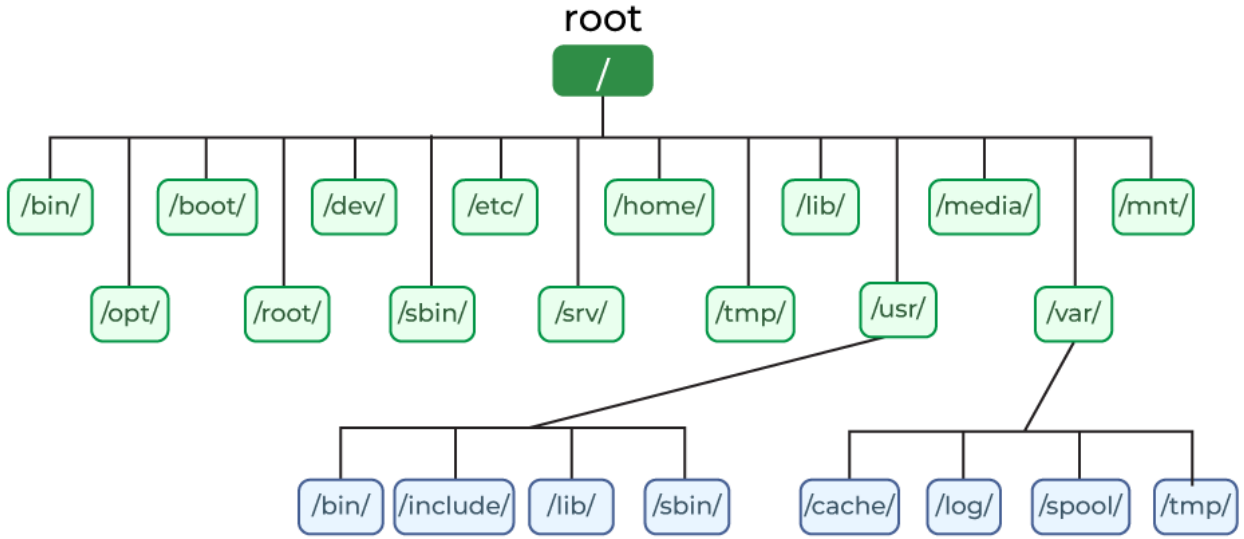
Unix File System - The Directory Tree

- A structured system for storing and organizing large amount of information for easy management.
- Everything is treated as a file. This includes:
  - Ordinary files that does not contain other files (data, text, program instructions)
  - Directories (equivalent to folders, branching points of the hierarchical tree)
  - Special files (represent real physical device such as a printer, disk, or terminal)
  - Etc.



Unix File System - Important Directories

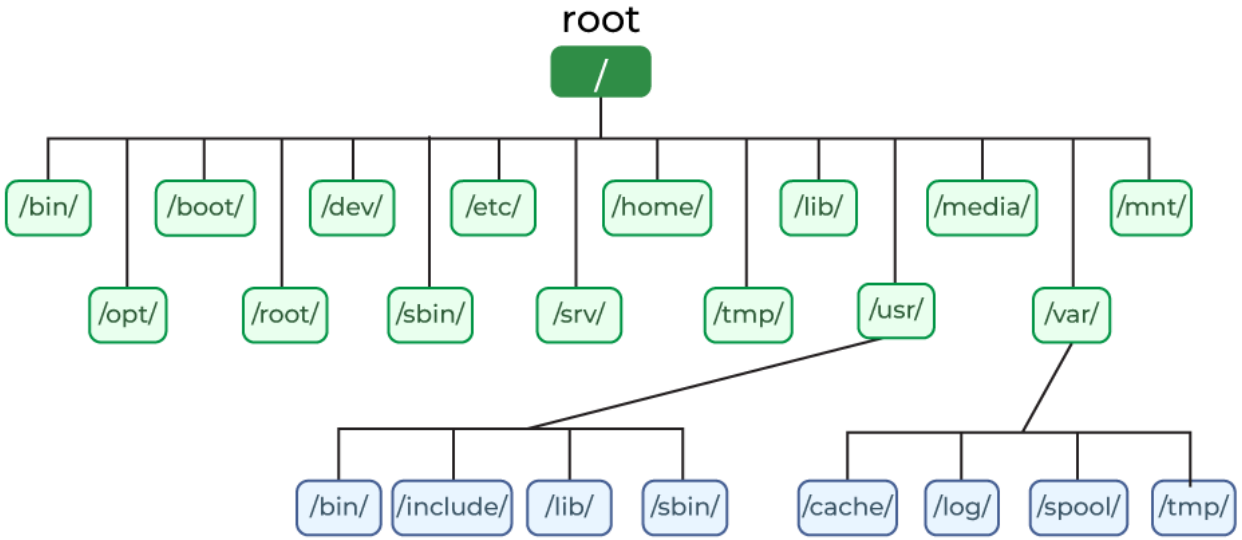
- **Root directory (/)**: the unique entry point for the whole file system. A *path* leading to a file or a directory can always be specified starting from the root directory.
- **/bin**: directory containing the major part of system commands.
- **/usr**: directory containing sub-directories with "user"commands.
  - **/usr/bin**: directory with frequently used commands.
  - **/usr/local**: directory with subdirectories containing tools installed on this specific machine.



- **/home**: top level directory of the user data directory tree.
- **/tmp**: a directory where anyone can read and write files (but, only a file's creator has the rights to remove her own stuff). Handy (though not necessary) to share data with colleagues on the same machine.

Unix File System - Important Directories

- **/project**: top-level directory for project directories, destined to hold project data than *needs to be backed-up* (initial data sets, results).
- **/scratch**: top-level directory of a directory tree mimicking the **/home** (unit/team/user) tree destined to hold "work" data such as those generated by ongoing computations and *not needing backups.* Files that are unmodified for the last 60 days will be *automatically removed on a monthly basis on the cluster.*

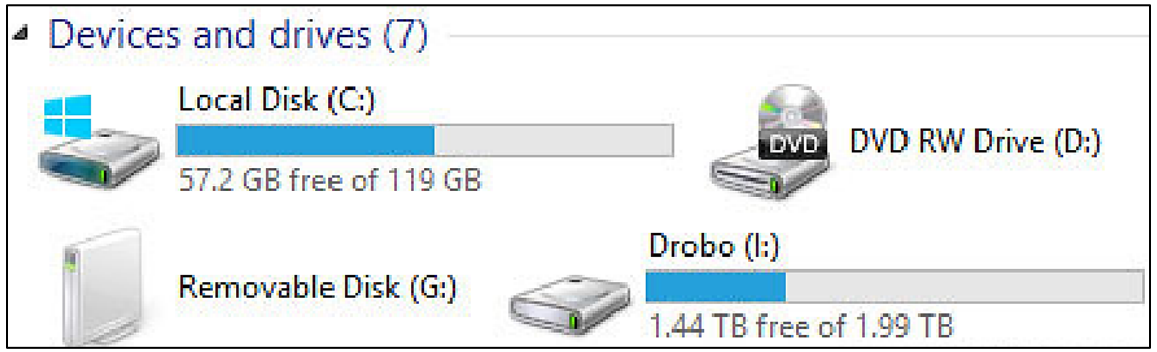


Please refer to https://www.cuhk.edu.hk/itsc/hpc/storage.html for the storage descriptions on CUHK computing.

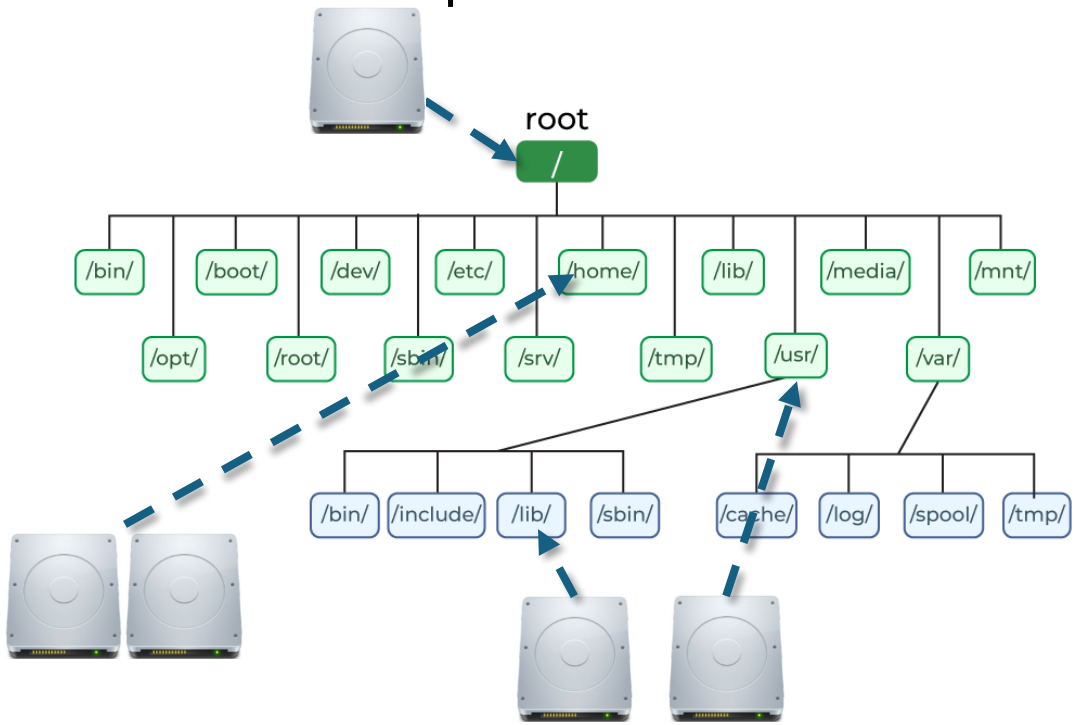
/lustre/project/eesc4550 is where we will base our work in throughout the course.

The File Systems in Comparison

- **Windows-like systems** define a single letter for each storage device (hard disk, DVD reader/writer, USB key, network drive), generating a **non-unique top-level directory tree**.
- In order to use a file, one has to know on which drive (letter) it is physically located.



- **UNIX/Linux like systems** associate all devices (not only storage devices), both local and remote, to directories in the **single rooted directory tree**, through so-called mount points.



Navigating in the File System

- Command execution takes place in the context of a **session**, defining at all times a **current user** (the user running the command) and a **current directory** or **working directory** (the directory where the command has been typed in). When starting a new session, the current directory is always the **home directory**.

Terminal

```
[s1155xxxxxx@chpc-logina ~]$ whoami # who am I (who is the current user?)
s1155xxxxxx
```

- The **pwd** command always prints an absolute path leading to the current directory.

Terminal

```
[s1155xxxxxx@chpc-logina ~]$ pwd     # print working directory
/users/s1155xxxxxx                   # the home directory
```

Navigating in the File System

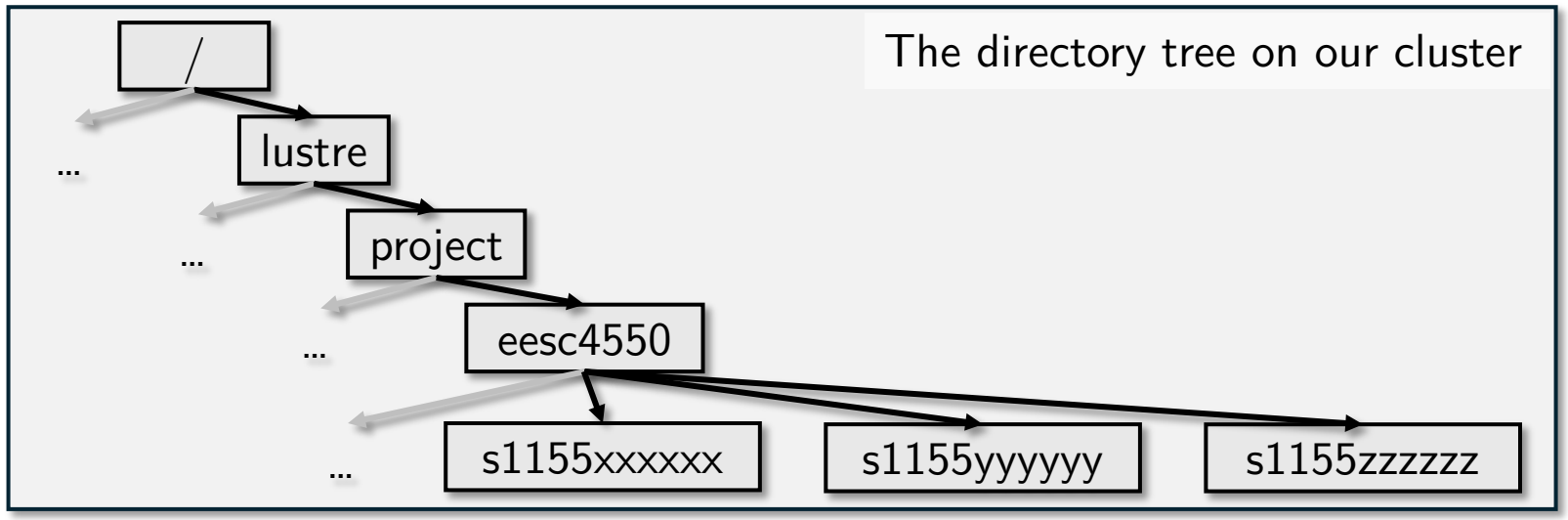
- To **c**hange the current **d**irectory:

Terminal

```
[s1155xxxxxx@chpc-logina ~]$ cd /lustre/project/eesc4550/
                # navigate to our course folder, using the absolute path
```

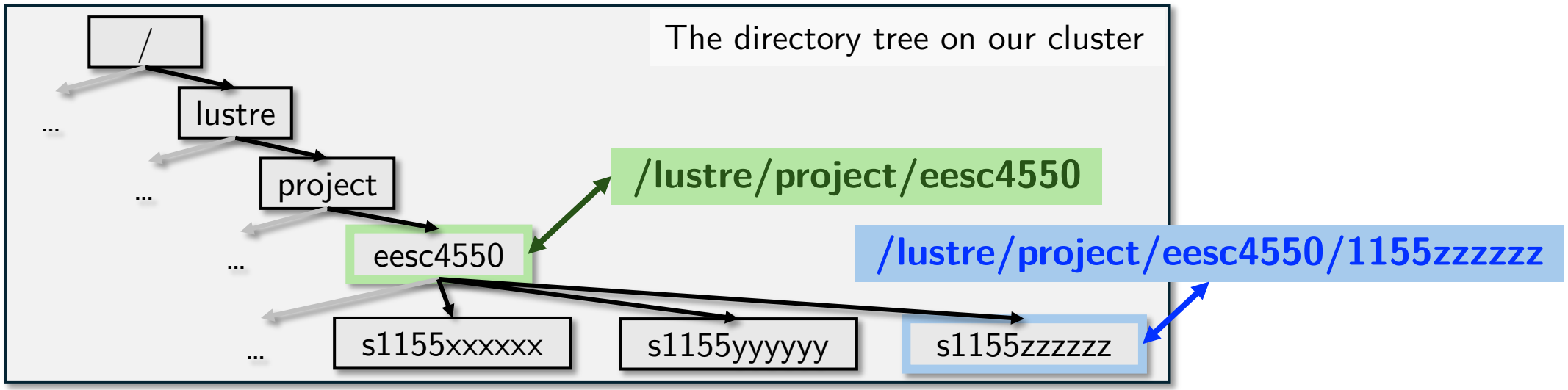
Terminal

```
[s1155xxxxxx@chpc-logina eesc4550]$ cd s1155xxxxxx
                # navigate to your personal folder, using the relative path
```



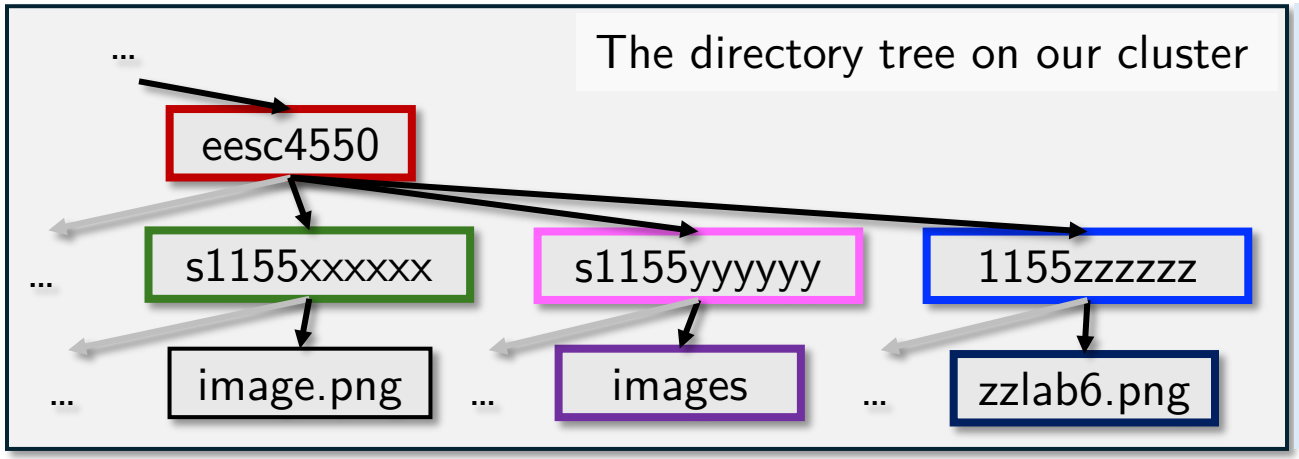
Navigating in the File System

- **Paths**: reference to file and directories.
- **Absolute** paths: built starting from the root directory and adding the subdirectories one by one separated by a slash character (/), until the desired file or directory is reached. *No matter what the current directory is, an absolute path will always lead to the same file or directory.*



Navigating in the File System

- **Relative** paths: built with the current directory as starting point and traversing the directory tree *upwards* or *downwards*, until the desired directory or file is reached. The successive path components are separated with slash (/) characters, and:
  - The dot ("**.**" character) always refers to the **current** directory
  - On each **upward** step in the tree, two dots ("**..**") are added to the path.
  - On each downward step in the tree, the name of the directory is added to the path.



The following *relative paths* all leads to **zzlab6.png**:

Starting from **1155zzzzzz**: **zzlab6.png**

Starting from **images**: **../../1155zzzzzz/zzlab6.png**

Starting from **eesc4550**: **s1155xxxxxx/./../1155zzzzzz/zzlab6.png**

Listing the Directory Contents

- Listing the contents of a directory, note that the current path does not change, even after a specific directory is specified.

Terminal

```
[s1155xxxxxx@chpc-logina ~]$ ls              # no arguments: current directory
KPP local
[s1155xxxxxx@chpc-logina ~]$ ls /users/s1155xxxxxx/local   # absolute path
bin  include  lib  share  src
[s1155xxxxxx@chpc-logina ~]$ ls KPP                        # relative path
AUTHORS.txt    CHANGELOG.md     ci-tests    docs         examples
LICENSE.txt    models           site-lisp   SUPPORT.md   bin
CITATION.cff   CONTRIBUTING.md  drv         int          log.kpp
README.md      src              util
```

Listing the Directory Contents

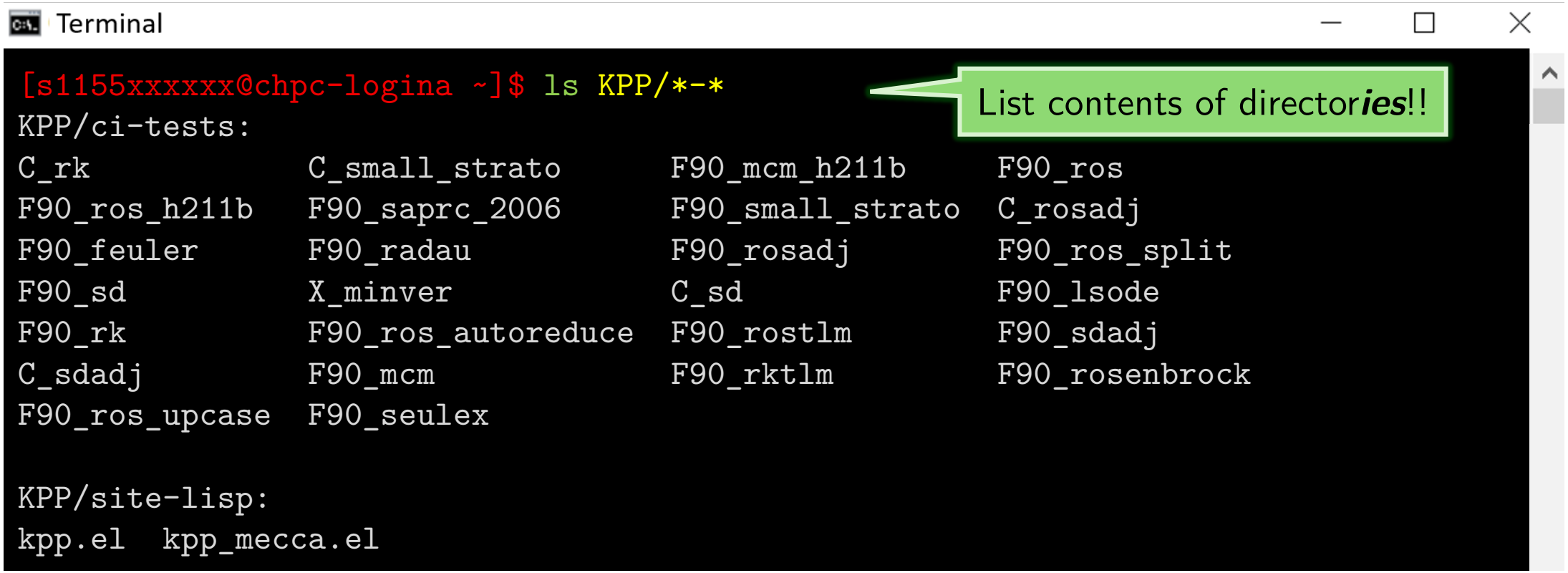
- By default, `ls` does not display files having names starting with a dot (aka "hidden" files / directories). The `-a` option needs to be added for them to be included.

Terminal

```
[s1155xxxxxx@chpc-logina ~]$ ls -a KPP
.                 CHANGELOG.md        CONTRIBUTING.md   .git
LICENSE.txt       .readthedocs.yaml   SUPPORT.md        ..
.ci-pipelines     docs                .github           log.kpp
.release          util                AUTHORS.txt       CITATION.cff
drv               .gitignore          models            site-lisp
.zenodo.json      bin                 ci-tests          examples
int               README.md           src
```

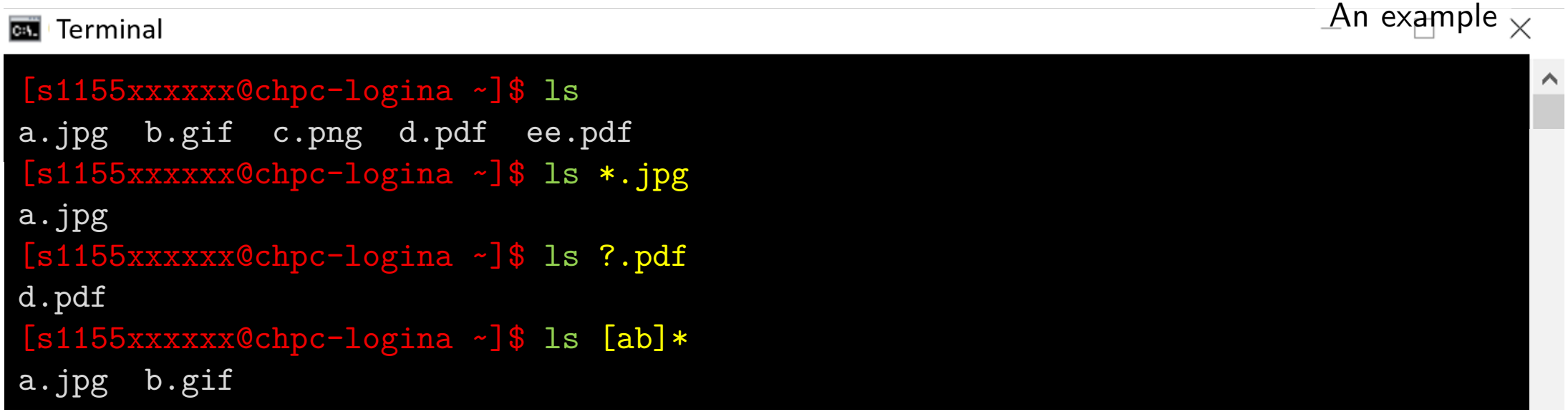
Pattern matching / Wildcards / Globbing

- Using **wildcards** in an argument of `ls` restricts the list to the files and directories whose names match the pattern formed by the argument.
- `*` matches zero or more characters.



Pattern matching / Wildcards / Globbing

- `?` matches any single character.
- `[...]` matches any of the characters in a set.



Creating a Folder

- The command `mkdir` **mk**es a new **dir**ectory.

Terminal

```
[s1155xxxxxx@chpc-logina ~]$ mkdir test1 test2
[s1155xxxxxx@chpc-logina ~]$ ls
KPP  local  test1  test2
```

- **Exercise.** Create the following directory structure in your workspace folder.

```
s1155xxxxxx
└── lec2
    ├── a
    │   └── d
    ├── b
    │   ├── e
    │   └── f
    └── c
```

Terminal

```
[s1155xxxxxx@chpc-logina ~]$ ls -aR
```

Copying Data

- The `cp` (**cop**y) command either renames or moves files and directories.
- Format: `cp SRC DEST` , where **SRC** is the path to the already existing data (the source data) and **DEST** is the path indicating the destination location.

Preparation. Go to your **lec2** folder, create a new file `lec2.txt` using `touch lec2.txt`.

Example 1. Copying a single file

```
cp lec2.txt lec2-copy.txt
```

Example 2. Copying a single file in another directory

```
cp lec2.txt a
```

Example 3. Copying a set of files to another directory

```
cp lec2*.txt b
```

Try listing the contents of the directory, **b**

Example 4. Copying a complete file structure

```
cp -r b ..
```

```
s1155xxxxxx
├── b
│   ├── e
│   ├── f
│   ├── lec2-copy.txt
│   └── lec2.txt
└── lec2
    ├── a
    │   ├── d
    │   └── lec2.txt
    ├── b
    │   ├── e
    │   ├── f
    │   ├── lec2-copy.txt
    │   └── lec2.txt
    ├── c
    ├── lec2-copy.txt
    └── lec2.txt
```

Move or Rename Data

- The `mv` (**mv**e) command either renames or moves files and directories.
- Format: `mv SRC DEST`, where **SRC** is the path to the already existing data (the source data) and **DEST** either the new name for the file or the directory in which it has to be moved.

Preparation. Navigate to your **lec2** folder.

Example 5. Renaming a single file

```
mv lec2-copy.txt lec2-edit.txt
```

Example 6. Moving a single file to an (existing) directory

```
mv lec2-edit.txt a
```

Example 7. Moving a set of files to an existing directory

```
mv lec2*.txt c
```

Example 8. Moving a directory structure

```
mv ../b g
```

Notice how **../b** reincarnate as **g**. What if **g** has already been created?!

```
s1155xxxxxx
└── lec2
    ├── a
    │   ├── d
    │   ├── lec2-edit.txt
    │   └── lec2.txt
    ├── b
    │   ├── e
    │   ├── f
    │   ├── lec2-copy.txt
    │   └── lec2.txt
    ├── c
    │   └── lec2.txt
    └── g
        ├── e
        ├── f
        ├── lec2-copy.txt
        └── lec2.txt
```

Deleting Data

- The `rm` (**rm**ove) command deletes the file(s) whose path(s) are given as argument.



What is deleted with `rm` CANNOT be restored.

Preparation. Navigate to your **lec2** folder.

Example 9. Removing a single file

```
rm b/lec2.txt
```

Example 10. Removing a set of files matching a pattern

```
rm g/lec2*.txt
```

Example 11. Removing a complete directory structure

```
rm -r g
```

```
s1155xxxxxx
└── lec2
    ├── a
    │   ├── d
    │   ├── lec2-edit.txt
    │   └── lec2.txt
    ├── b
    │   ├── e
    │   ├── f
    │   └── lec2-copy.txt
    └── c
        └── lec2.txt
```

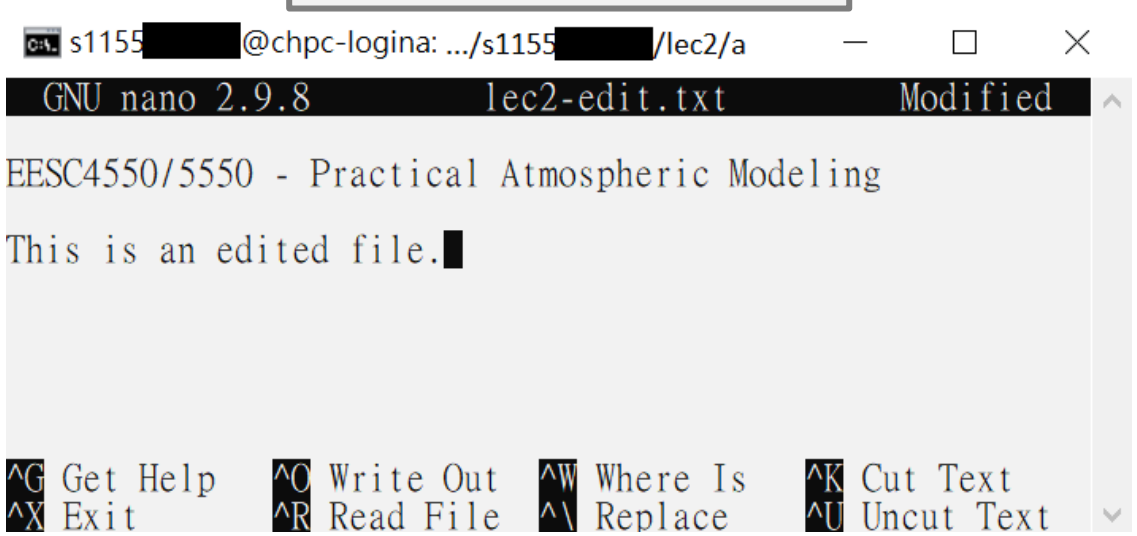
File Names

- Linux is **very permissive** about valid characters in file. It's safer to follow the community conventions and stick to the alphanumeric characters.
  - ***Case sensitive; Hidden files start with a "."***;
  - ***Recommended***: lowercase characters, digits, underscore, dash;
  - ***Unrecommended***: Space characters, leading hyphens, and other punctuation marks.
- Linux **doesn't put any requirements on file name extensions** (`.txt`, `.csv`, `.pdf`, `.html`, etc.). Any extension can be given to any type of file, but it is recommended to keep it consistent with the file content for readability.

Changing the Contents of a (Text) File

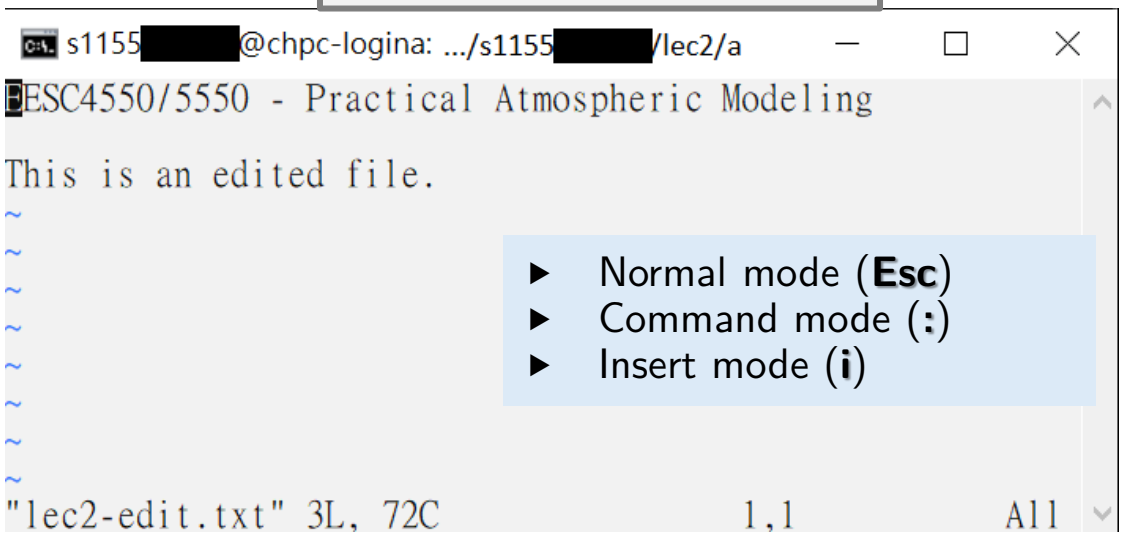
- Two text editors: ***nano***, ***vim***
- Learn more: ***nano*** cheatsheet; ***vim*** cheatsheet

`nano a/lec2-edit.txt`



- **Lightweight, beginner-friendly**
- **Save:** `Ctrl` + `O`
- **Exit:** `Ctrl` + `X`

`vim a/lec2-edit.txt`



- Normal mode (**Esc**)
- Command mode (**:**)
- Insert mode (**i**)

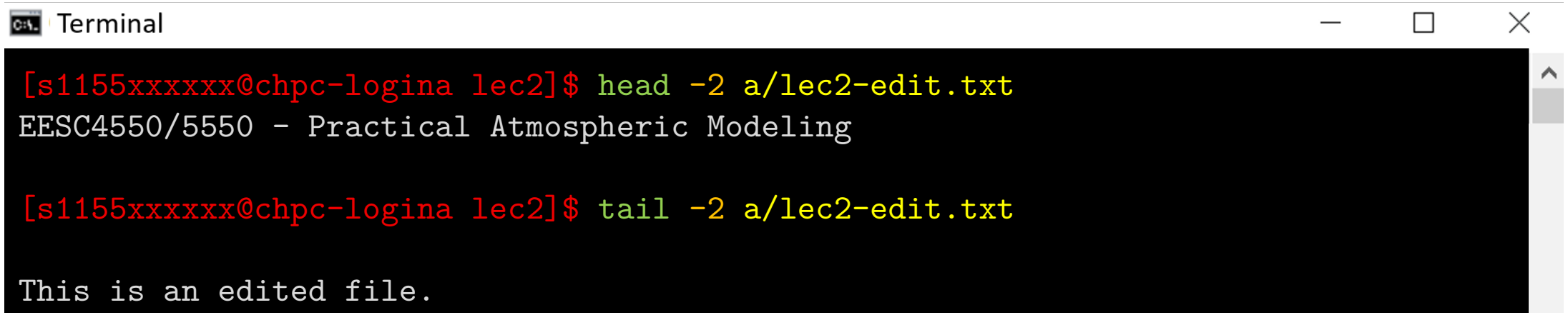
- **Multi-modal, handles huge files well, for the tech-savvy** 🤓
- **Save & Exit:** `Esc` + `:wq`
- **Exit w/o save:** `Esc` + `:q!`

Examining the Contents of a File

- Displaying the ***entire*** contents of a file: `cat`

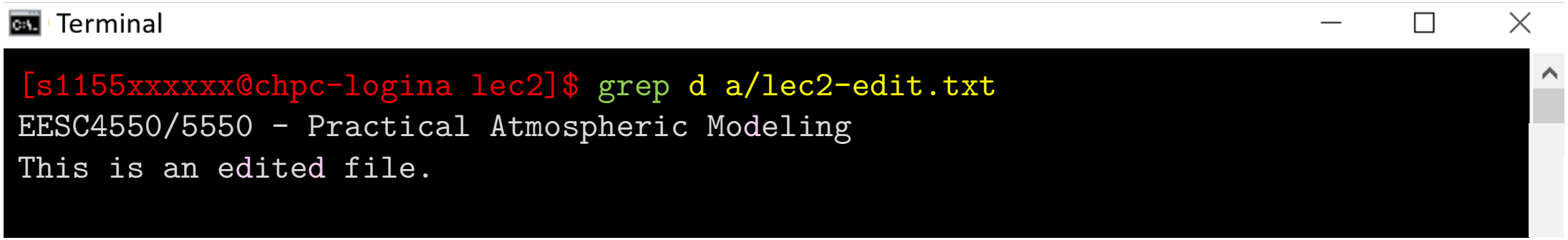


- Displaying the ***first***, and then the ***last*** few lines of a file: `head` `tail`



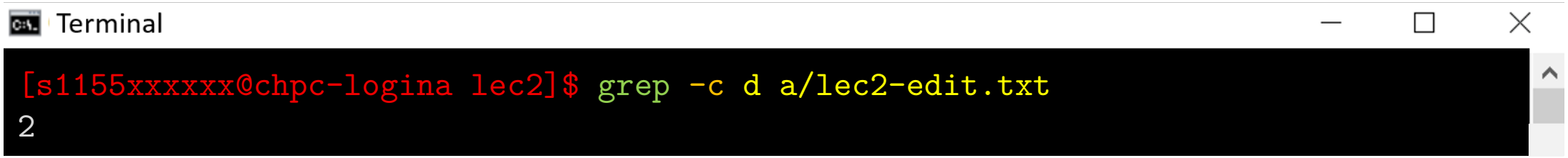
Searching for Information in a File

- The `grep` command takes two arguments: a ***pattern*** and a ***file name***, and it displays every line of the file containing the pattern the ***entire*** contents of a file:



Exercise 1. How to override the case-sensitive behavior of `grep`?

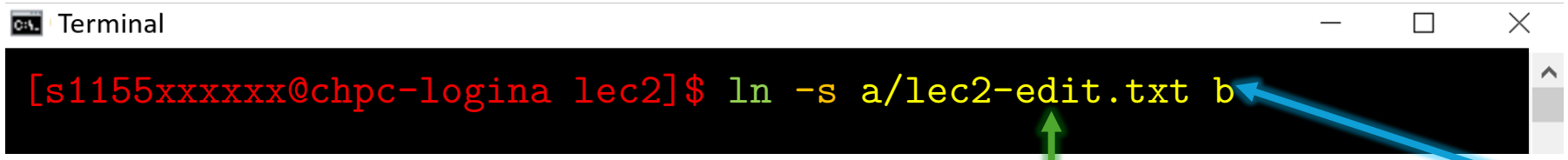
- `grep` also allows us to count the lines matching the pattern with `-c` .



Exercise 2. How to search for a pattern in all files of a directory tree?

Symbolic Links

- It is handy to provide an alternative access path to the target's content ***without duplicating*** it – also to save storage space!!
- Symbolic links (soft links) stores a path that refers to another file or directory.




```
s1155xxxxxx
└── lec2
    ├── a
    │   ├── d
    │   ├── lec2-edit.txt
    │   └── lec2.txt
    ├── b
    │   ├── e
    │   ├── f
    │   ├── lec2-copy.txt
    │   └── lec2-edit.txt -> a/lec2-edit.txt
    └── c
        └── lec2.txt
```

First argument: path of the existing file

Second argument: the directory to create the shortcut with the same name

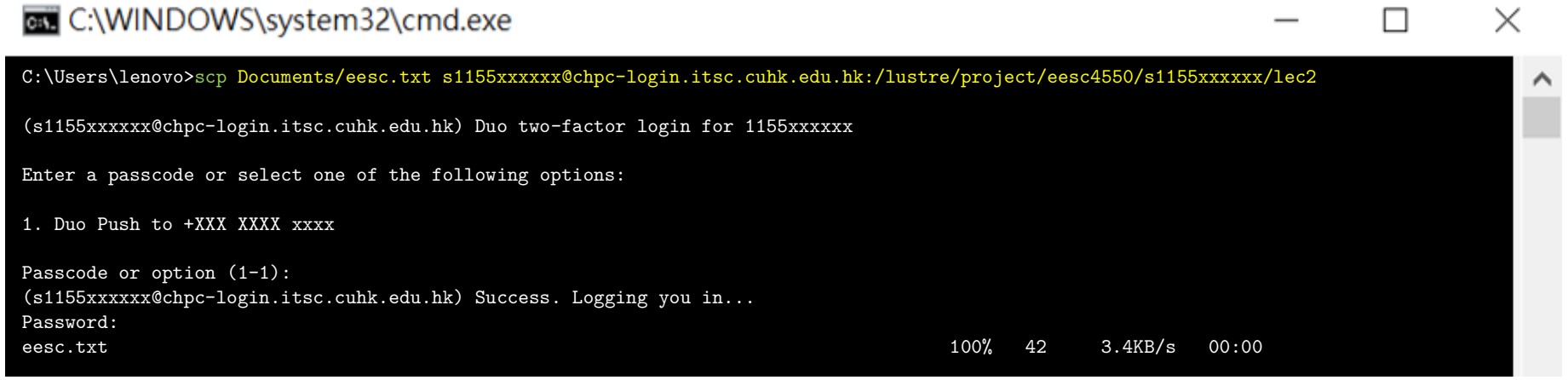
Note: Deleting the original file makes the shortcut unusable!

File Transfer

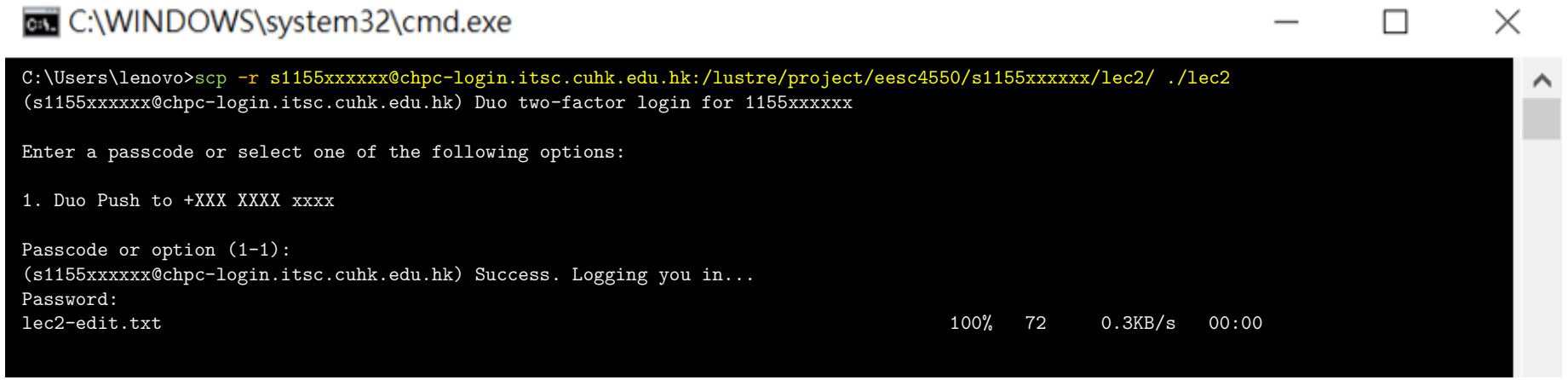
- The `scp` (**S**ecure **C**o**p**y) command is used to copy files to/from another Linux (UNIX) machine. To use it, it's necessary to have an account (username and password) on the remote machine.
- `scp` is used like `cp` but one of its arguments (source or destination) includes information about the remote machine.
- **Example Usage**: `scp SRC username@hostname:DST`

File Transfer - Exercises

- **Exercise 1**. Copying a local file to a remote machine.

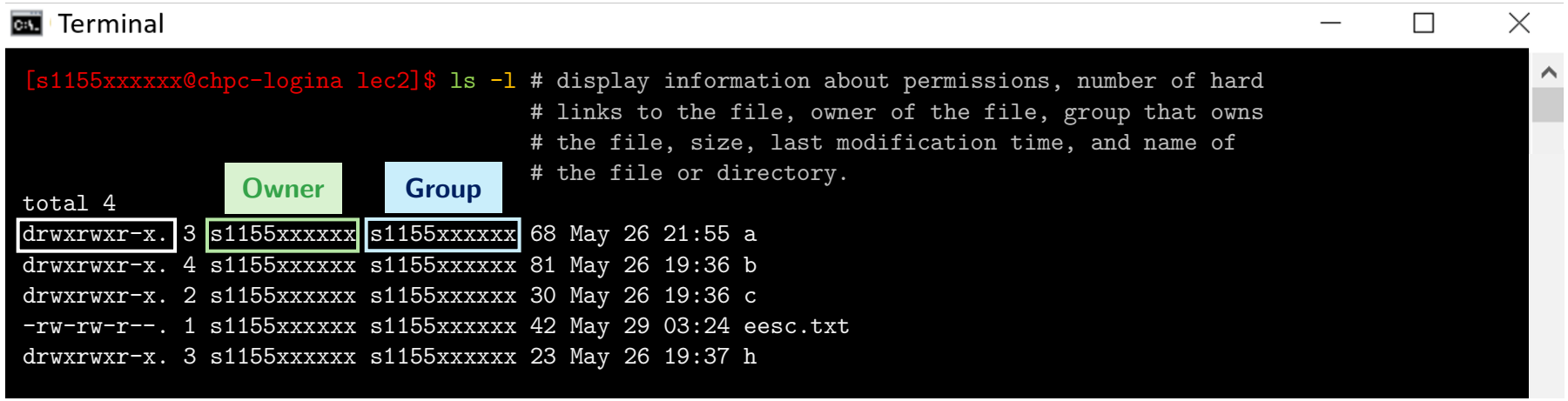


- **Exercise 2**. Copying a directory structure from a remote machine.

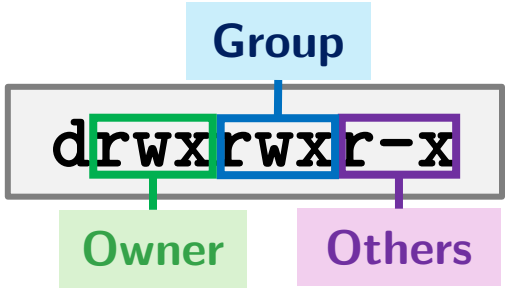


Users and Groups

- Each resource (file, directory, running program...) is owned by a user (the "***owner***") having a valid account on the machine.
- Users and groups have numerical identifiers, called `uid` (user id) and `gid` (group id).
- Every user is limited to what he/she has access to on the system: file read/write access, program execution, allocated disk or memory space.
- The `root` user has no access limitations.
- Access rights are grouped in ***triplets*** made of the `r`, `w`, `x` or `-` characters.



Managing Access Rights



Position in the triplet	Character	Access right for a file	Access right for a directory
1	`r`	Read access allowed/forbidden	Reading the list of files is allowed/forbidden.
2	`w`	Write access allowed/forbidden	File creation, renaming and removal are allowed/forbidden
3	`x`	Execution access allowed/forbidden	Changing to this directory is allowed/forbidden.

`-` forbids the access corresponding to its position in the triplet.

- The `chmod` command is used to modify access rights to files and directories. Its first argument defines the access right modifications to apply. Its second argument defines the file(s) or directory(ies) on which to apply the modifications.
- The command has an `-R` (recursive) option recursively applying the access rights to all files and subdirectories of its destination argument.

Managing Access Rights - Exercises

- **Example 1**. Make a file "private" by removing all rights for the group and others.

```
chmod go-rwx lec2.txt
```

- **Example 2**. Add write access for the group to a file.

```
chmod g+w lec2.txt
```

- **Example 3**. Prevent file modification to everyone.

```
chmod -w lec2.txt
```

- **Example 4**. Add execution rights to a program.

```
chmod +x program
```

- **Example 5**. Change the group owner for a file, the user running the command must be a member of the group.

```
chgrp +x group lec2.txt
```

Processes

- A process is a ***currently running program***. Each time a user issues a command (runs a program), the operating system loads it into memory and starts its execution.
- In order to run smoothly, a process needs ***memory and an allocated processor time***. It is the duty of the operating system to proceed to the optimal allocation of these resources among all the processes running "simultaneously". The load of a machine reflects the activity of all the active processes at any given moment.
- The user can ***stop the process*** execution "by force", ***interrupt them*** to resume them later, or ***modify their priority***.

Running and Terminating a Program

- A process running in the current session can be (brutally) ***stopped*** by Ctrl + C .
- A process running in the current session can be ***interrupted*** with by Ctrl + Z .

example.py

```
import time
i = 0
while True:
    print(i)
    i += 1 # increments i by 1
    time.sleep(1) # pause execution
```

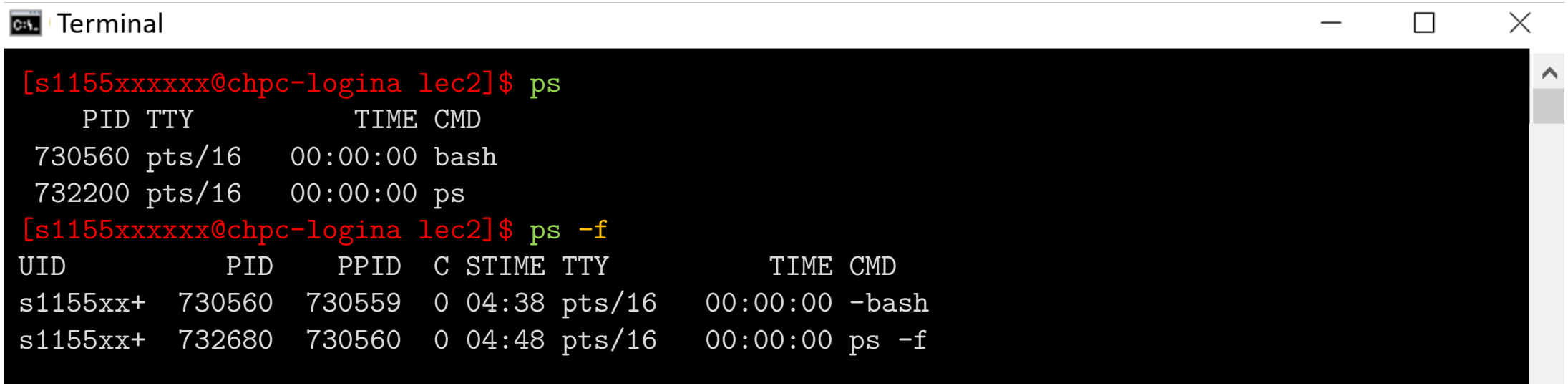
Terminal

```
[s1155xxxxxx@chpc-logina lec2]$ python2 example.py
0
1
2
^CTraceback (most recent call last):
  File "example.py", line 6, in <module>
    time.sleep(1) # pause execution
KeyboardInterrupt
[s1155xxxxxx@chpc-logina lec2]$ python2 example.py
0
1
^Z
[1]+  Stopped                 python2 example.py
```

Job identifier

Process Information

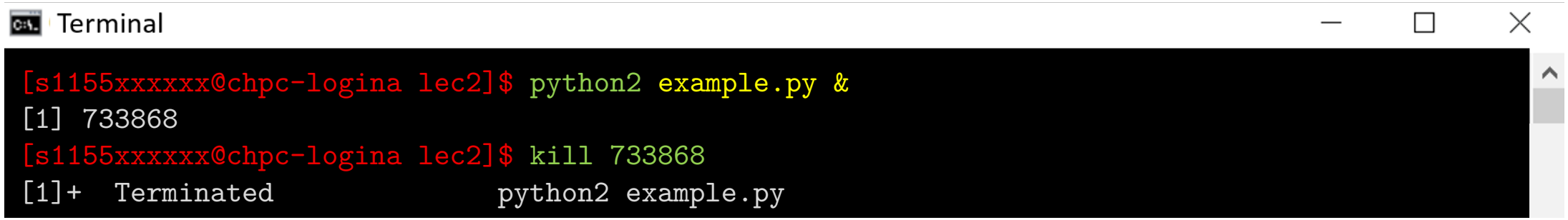
- The `ps` (**p**rocess **s**tatus) command is used to display information about processes.



- By adding the option `-f`, additional information is shown about the user (`UID`), the PID of the parent process (`PPID`), the start time (`STIME`), the execution time (`TIME`) and the complete command line (`CMD`).

Killing a Process

- The `kill` command sends a signal to the ***user's own process*** whose process identifier (PID) is given as argument.



- Every process is spawned from a parent process. For instance, opening a session yields a new process, which will be the parent process of all of the commands typed in on the command line.
- The process with PID 1 is the process that, when the machine was started, gave rise to the system's process tree.
- Halting a process results in halting all of its child processes.

Submitting a Job to Slurm

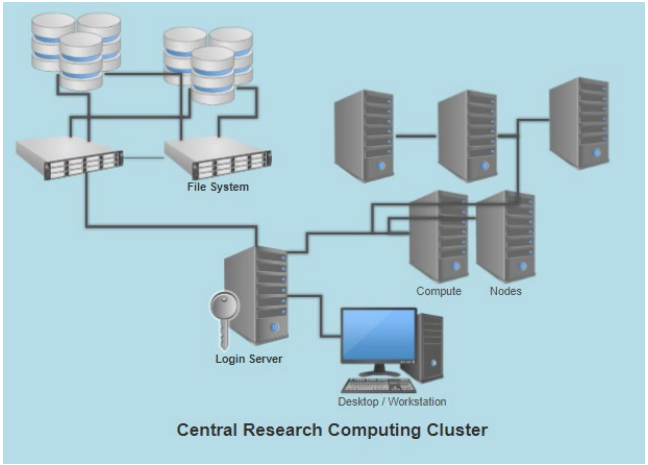
- Slurm is an open source cluster management and job scheduling system. It queues and allocate jobs to run on compute nodes based on the resources available and the resources specified in the job script.

job.slurm

```
#!/bin/bash

#SBATCH --job-name=singlecputasks
#SBATCH --ntasks=2
#SBATCH --cpus-per-task=1

# Your script goes here
srun --ntasks 1 python2 example.py
srun --ntasks 1 echo "I'm task 2"
```



Terminal

```
[s1155xxxxxx@chpc-logina lec2]$ sbatch job.slurm
Submitted batch job 208111
[s1155xxxxxx@chpc-logina lec2]$ squeue --job 208111
[s1155xxxxxx@chpc-logina lec2]$ scancel 208111
```

Simpler Code Development on VS Code (Optional)

- In this course, we will create many scripts for visualization of information or modify configuration files from numerical models.
- A ***graphical user interface*** is often simpler for us to operate on.
- **Download VS Code**: https://code.visualstudio.com/
- **Install OpenSSH Server (for Windows users)**: https://learn.microsoft.com/en-us/windows-server/administration/openssh/openssh_install_firstuse?tabs=powershell&pivots=windows-10

- Instruction: https://code.visualstudio.com/docs/remote/ssh#_getting-started

Installing the Remote SSH Extension (Optional)

2 Search "remote-ssh"

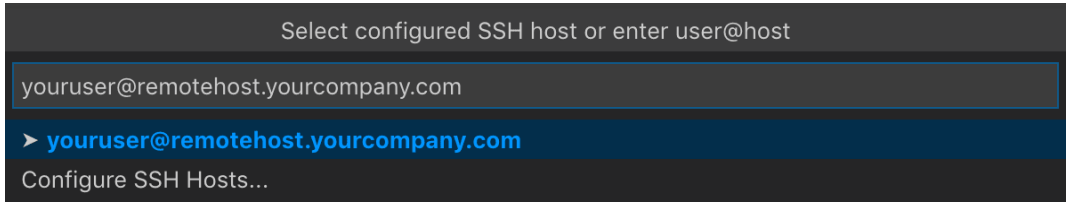
1 Extensions

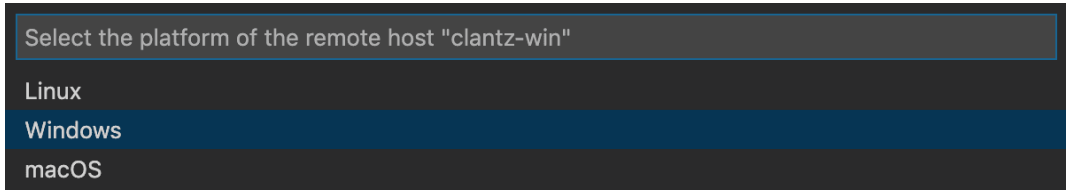


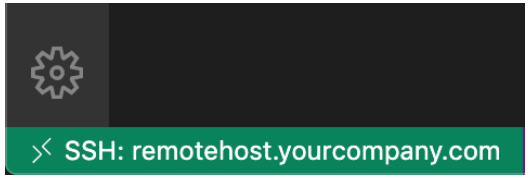
3 You should be able to install the extension from here.

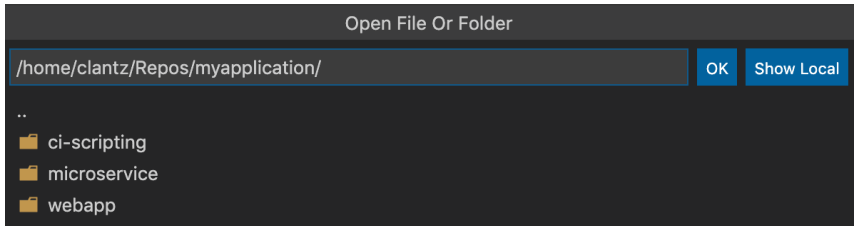
Connecting to a Remote Host

- In VS Code, select **`Remote-SSH: Connect to Host...`** from the Command Palette (**`F1`**, **`Ctrl`** + **`Shift`** + **`P`**) and enter the same `user@hostname`.
- If VS Code cannot automatically detect the type of server you are connecting to, you will be asked to select the type manually (select ***Linux***).
- You will be asked for the login information. After you are connected, you'll be in an empty window. You can always refer to the Status bar to see which host you are connected to.
- You can then open any folder or workspace on the remote machine using **`File > Open...`** or **`File > Open Workspace...`** just as you would locally!

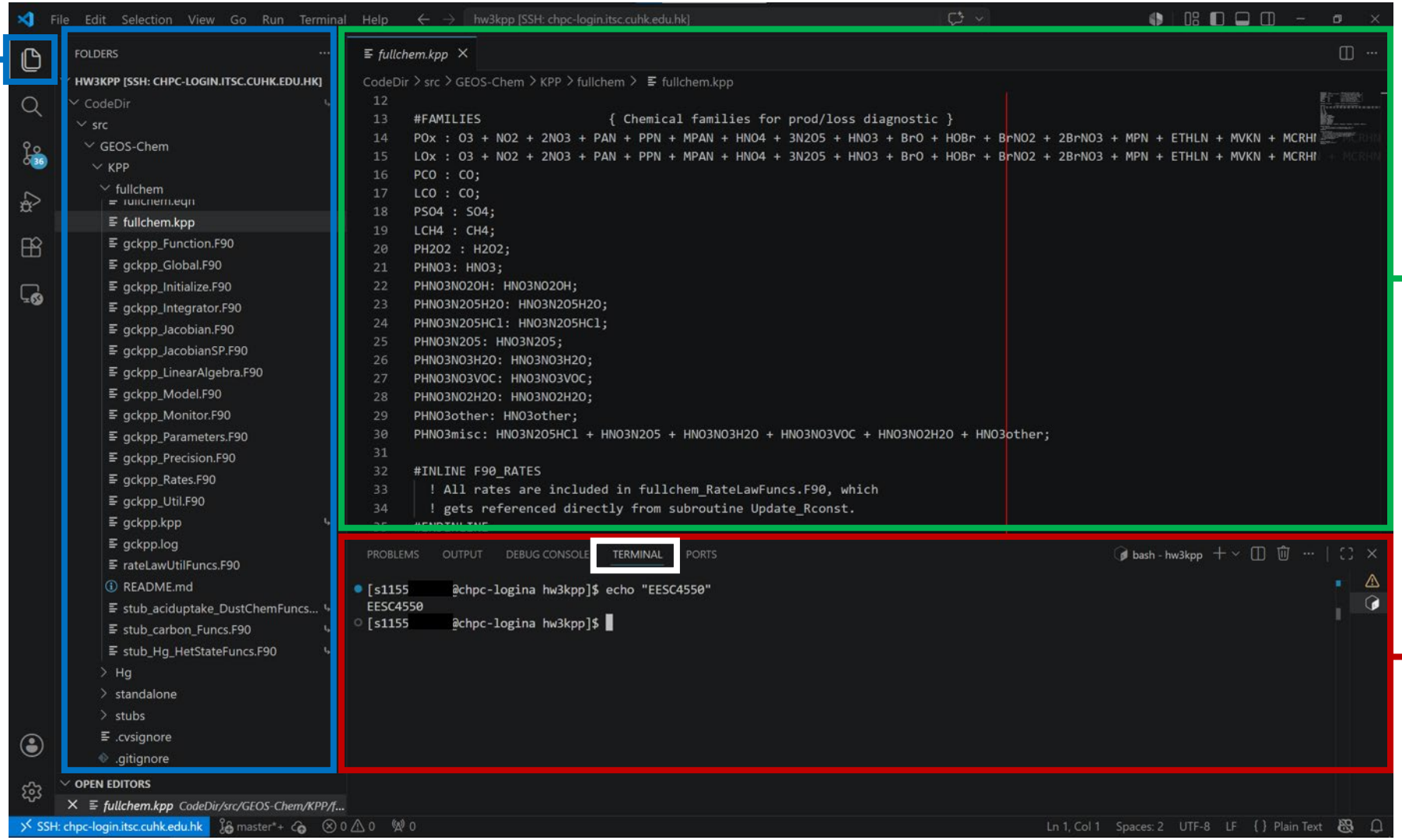








Navigating in VS Code



Lab Exercises

- **Goal for this week**: Practice Linux command operations on the central cluster. You will also explore how to create a basic script that does a sequence of commands ***all at once***.

- ***Avoid plagiarism at all cost.***

> *Plagiarism* is presenting someone else's work or ideas as your own, with or without their consent, by incorporating it into your work without full acknowledgement.

References

Hoebeke, M., & Aroumougom, J.-M. (2022). *Linux for Beginners* [Licence]. Station Biologique de Roscoff, France. https://hal.science/hal-04029594v1

Wakelin, J., Gretton, L., Gilchrist, G., & Forey, T. (n.d.). *Linux Tutorial* (Version 1.21). University of Leicester. Adapted from M. Stonebank, *UNIX Tutorial for beginners*.